

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Artificial Intelligence and Data **Science**
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Slot-c

34

Course Code:

CSA140

Course Title:

COMPILER DESIGN

CO Assessed:

CO1 - Imitation (L1)

Assessment:

Assessment Tool 1 - ANALYTICAL PROBLEM SOLVING

Weightage:

40% of **CIA**

CO1 Statement:

Apply lexical analysis **for** token specification and recognition using LEX tool.

(BL3)

Student Name:

Section / Batch:

V.Vamsidhar Reddy

Reg. No.:

Date:

192365029

16/07/26

Code of Conduct

192365029

I V. Vamsidhar Reddy / *Name/Reg No*) *certify that this submission is my original work and that I have*

adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Vanni

Instructions

Develop a modular and **efficient** Python program that satisfies **all** the functional requirements **specified** in the problem statement.

Demonstrate the correctness **of** the implementation using suitable test cases and justify the **output obtained**.

Follow good programming practices, including meaningful variable names, proper code **indentation**, comments, and error handling wherever applicable.

Duration: 30 minutes.

Q.

Problem

No

Method

Understanding (20) Selection (20)

Solution Process (25)

Accuracy of Calculation

Interpretation of Results

(20)

(15)

Total Score (out of 100)

1
4
/4
4 1 4
4/4
4
1 4
3 1 4
96.25

2
4/4
3 1 4
3/4
4 1 4
/4

3
4
3 1 4
/4
3/4
4/4
1 4
4 1 4

4
88.75
88.75

/4
4
14

3/4

14

3

4/4

5

3/4

314

/4

/4

9/4

88.75

90

OVERALL RUBRICS VALUE

A

/4

/4

/4

/4

/4

96.5

/ 20

/20

/25

/20

/ 15

/100

N . DH

Annexure A

ANALYTICAL PROBLEM SOLVING

1 Trace the processing of the input expression $P = (a^* b) + (bc)^*$ through all phases of a compiler.

a) Identify and list all tokens generated during lexical analysis.

b) Construct the syntax tree during the parsing phase.

c) Generate the intermediate code for the given expression.

d) Apply suitable optimizations to the intermediate code.

e) Explain the significance of the final target code produced.

2 For each of the following Regular Expressions, construct the language. Write **any** FIVE valid strings for each Regular Expression. a) $(\epsilon + aa^*)^*$

b) $(a^* + b^*)^* abb$

c) $(a^* b^*)^* aba (a^* + b^*)^*$

d) $(a + ab)^*$

e) $(aba)^* + (a^* b^*)^*$

3 i) Find the number of tokens in the following C statement is

```
main() {  
    int a=10;  
    char b="abc";  
    int c = 30;  
    char d = "xyz";  
    in /* comment */ t m = 40.5;  
}
```

ii) Identify the lexical errors in the following C statement is

```
void main()  
{  
    int x=10, y=20;  
    char * a;
```

```
a= &x;  
x= 1xab;
```

4 Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$:

- i) The language of all **strings** that have an **even** number of 0's.
- ii) The language of all strings that contain at **least one** 1.
- iii) **The language of** all strings in **which** both the number of 0's and the number of 1's are odd.
- iv) **The language of** all strings that start with 1 and **end with 0**.
- v) The language of **all** strings that do not contain consecutive 1's.

5 Consider a lexical analyzer for a simplified programming language. The language consists

of the following tokens:

- 1. Keywords: if, else, while, return, int, float
- 2. Operators: . . . / . .
- 3. Separators: (,), {, }, ..
- 4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123

The

5. Integer Literals: A sequence of digits, e.g., 12345

6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of

digits, e.g., 123.45

lexical analyzer

needs to tokenize the

following input string:

```
int var123 = 12345; float var456 = 123.45; if (var123 var456) { return; }
```

Questions:

a)

How many tokens are there in the given input string?

b)

Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

c)

Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of **state transitions** required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

CSA1407

Compiles phases:

Expression :

$P = (a * b) + (b * c) * 2$

V.Vamsidhar Reddy

192365029.

(a) Tokens (Lexical Analysis):

Token

P

1

C

a

*

b

)

+

(

b

*

c

)

*

2

TYPE Identifier

Assignment operator Left Parenthesis.

Identifier

Multiplication operator

Identifier

Right parenthesis Addition Operator Left
parenthesis

Identifier

Multiplication operator

Identifies

Right parenthesis

Multiplication operator Integer constant

In Lexical Analysis,

Total Tokens = 15.

The expression contains 15 tokens.

Q

(1) Syntax Tree:

P

*

+

*

a
b *
2

b * c

(C) Intermediate Code :-

t1 = a * b

t2 = b * c

+3 = +2*2

+4 = t1++3

p = ty

(d) optimized Intermediate Code:-

t1 = a*b

t2 = b*c* 2

P = +1++2

@Significance of Target

Code:- Target code is the final machine/ assembly

Code generated by the compiler. It can be executed directly by the processor and is optimized for efficient execution.

Regalos Expoession:

$$(a) \varepsilon aa^*)^* = (\varepsilon + aa^*)^*$$

Language: Any number of groups of one

O21 more a's.

Valid Strings : -

1. & (empty String) - Epsilon

g.a

3.aa

4.aaa

5.aaaaa

(b) $(a^*b^*)^*abb.$

**Language: Strings ending with
abb.**

Valid Strings:-

1. abb

2. aabb

3. babb

4.aaabb

5. ababb

(c) $(a^*b^*)^*aba(a^*+b^*)^*$

Language: Strings containing the substarling

aba.

Valid Staings:-

1. aba

2. abaa

3. ab ab

5.baaba

4. aaba

(d) $(aab)^*$

Language : Ledio or more grepetitions of aab.

Valid Strings : —

• ϵ (empty string) -

Epsilon

2. aab

3. aabaab

4. aabaabaab

5. aabaabaabaab

$(\epsilon \text{ aba})^* + (a^*b^*)^*$

Language: union of $(\text{aba})^*$
and $(a^*b^*)^*$

Valid Strings: -

1. ϵ (empty string) -

Epsilon

2. aba

3. aaabbb

4. aaaa

5. bbbb.

(3) (1) Number of Tokens:..

Program:-

Main ()

{

int a = 10;

char b = "abc";

int c = 30;

if (x < y)

chard = "XYZ";

in /* comment */ &m=40.5;

3a

Tol

Tokens:- ↓

main

"abc"

30

40.5

10

;

;

;

;

)

₹

es

;

in

ch

in

3

Char

t

05

.t"

int

b

J

d

3

a

=

. . . Total Token = 33

(ii): Lexical Errors:—

```
void main ( )
```

```
int x = 10; 4 = 20;
```

```
char *a;
```

```
a = 4 x ;
```

```
}
```

```
x=1x ab;
```

Errors:

1. Ixab→

Invalid Hexadecimal constant

2. `A=&x;` →

No lexical error

Only

a Semantic `type` error).

C Regular

Expressions:-

5

(i) Even number of 08 :

→ $1^* (01^* 01^*)^{**}$

(ii) Atleast one 1:

↑

$(0/1)^* (0/1)^*$

(iii) Odd number of 0's and odd number of 1's:

$1^* (01^*01^*)^*01^* (10^*10^*)^*0^*$

(IV) Starts with 1 and ends with 0:

1 2 1

$1(011)^*0$

(V) No Consecutive 1's:

→ $(0/10)^* 1 ?$

6 Lexical Analyzer:-

Input:- Ponogram-

```
int Noor 123 = 12345; float Var 456 = 123.45;
```

```
if ((var123) == \ar 456)
```

سمى

```
return;
```

```
}
```

RUBRICS FOR CALCULATION:

Criteria

Weightage

Level 4

Exemplary

Level 3 Proficient

Level

2

Level

Developing

Problem

20%

Clearly

Understands

Partial

Understanding

interprets

problem with understanding,

1

Beginning

Misinterprets

or unable to

problem,
minor gaps
misses

identifies all
elements
key understand
problem

parameters
and
requiremen
ts
accurately

Method Selection 20%

Selects

Selects

Inappropriate or Unable

most
correct
partially
correct
to

select method

appropriate
method
with
method

method/**for**
minor **errors**

mula

with

strong

justification

Solution Process

25%

Logical,

systematic

Mostly correct Disorganized

steps with steps; multiple **process**

steps with minor

errors

No

clear

Accuracy Calculation

correct

mistakes

approach

throughout

of 20%

All

Minor

calculations

calculation

Frequent errors;

errors; No **incorrect** final

accurate;

errors

answer

Interpretation **of** 15% Results

correct final

answer

Clearly interprets

results with

Basic

Limited

interpretation unclear

with minor interpretation

units and omissions

engineering

relevance

valid

calculations

or No

interpretation